

Client Interaction (Browser → Server Journey) — One-Page Revision Notes

1. Complete Request Flow

```
graph TD; User --> Browser; Browser --> Browser_Cache[Browser Cache?]; Browser_Cache --> DNS_Lookup[DNS Lookup]; DNS_Lookup --> IP_Address[IP Address]; IP_Address --> TCP[TCP 3-Way Handshake]; TCP --> TLS[TLS Handshake (HTTPS)]; TLS --> HTTP_Request[HTTP Request]; HTTP_Request --> Load_Balancer[Load Balancer]; Load_Balancer --> App_Server_1[Application Server]; App_Server_1 --> Cache; Cache --> Cache_Hit[Cache Hit?]; Cache_Hit --> Return_Response[Return Response]; Cache_Hit --> Cache_Miss[Cache Miss]; Cache_Miss --> Database; Database --> App_Server_2[Application Server]; App_Server_2 --> HTTP_Response[HTTP Response]; HTTP_Response --> Browser_Rendering[Browser Rendering];
```

User
|
Browser
|
Browser Cache?
|
DNS Lookup
|
IP Address
|
TCP 3-Way Handshake
|
TLS Handshake (HTTPS)
|
HTTP Request
|
Load Balancer
|
Application Server
|
Cache
|
Cache Hit? → Return Response
|
Cache Miss
|
Database
|
Application Server
|
HTTP Response
|
Browser Rendering

2. Components

Client

- Browser, Mobile App, Desktop App
- Sends requests

Server

- Processes requests
- Returns responses

DNS

- Converts Domain Name → IP Address

Example

```
google.com
  ↓
142.xxx.xxx.xxx
```

TCP

- Reliable communication
- Guarantees packet order
- Uses Three-Way Handshake

```
SYN
↓
SYN + ACK
↓
ACK
```

TLS

Provides:

- Encryption
- Authentication
- Integrity

Remember:

HTTPS = HTTP + TLS

HTTP

Application layer protocol

Common Methods

```
GET
POST
PUT
DELETE
PATCH
```

HTTP is **Stateless**.

3. Request Structure

```
URL
Method
Headers
Body (optional)
```

Example

```
POST /login

Headers

Authorization
Content-Type

Body

{
  email,
  password
}
```

4. Response Structure

Status Code

Headers

Body

Example

200 OK

```
{  
  "name": "John"  
}
```

Common Status Codes

Code	Meaning
200	Success
201	Created
301	Redirect
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

5. Cookies vs Sessions vs JWT

Cookies

- Stored in Browser
- Small data
- Client-side

Sessions

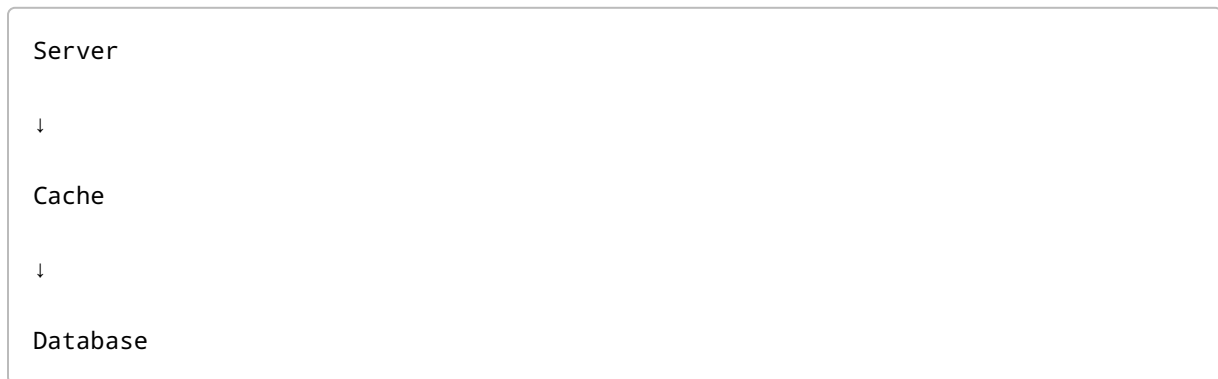
- Stored on Server
- Browser stores Session ID
- More secure
- Requires server lookup

JWT

- Stored on Client
- Server verifies signature
- No server-side session storage
- Preferred for Microservices

6. Cache

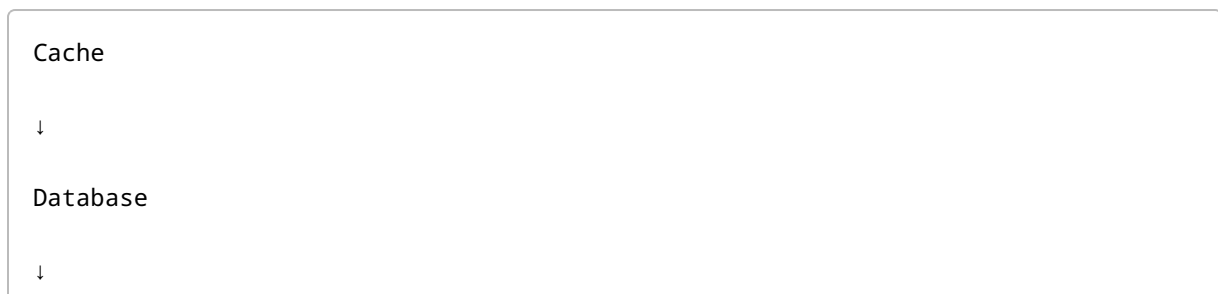
Purpose



Advantages

- Faster Response
- Lower Database Load
- Reduced Latency

Cache Miss



Update Cache

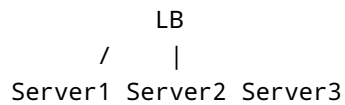
↓

Return Response

7. Load Balancer

Purpose

Distributes traffic across multiple servers.



Benefits

- High Availability
- Fault Tolerance
- Better Scalability

8. Browser Cache

Stores

- Images
- CSS
- JavaScript
- Fonts

Benefits

- Faster page load
 - Less bandwidth
 - Reduced server requests
-

9. DNS Cache

Browser remembers DNS result until TTL expires.

Benefits

- Faster lookup
 - Fewer DNS requests
-

10. Keep-Alive

Without Keep-Alive

Connection

↓

Close

↓

Reconnect

With Keep-Alive

One Connection

↓

Multiple Requests

Benefits

- Lower latency
 - Fewer TCP handshakes
-

11. Connection Pooling

Instead of opening a new database connection every time,

Application

↓

Connection Pool

↓

Database

Benefits

- Faster
- Less overhead
- Better throughput

12. Server Side Rendering (SSR)

Server generates HTML.

Benefits

- Better SEO
- Faster First Paint

13. Compression

Common Algorithms

- Gzip
- Brotli

Benefits

- Smaller payload
 - Faster downloads
 - Lower bandwidth usage
-

14. HTTP/2

Features

- Multiplexing
- Header Compression
- Stream Prioritization

One TCP connection handles multiple requests.

15. HTTP/3

Uses QUIC over UDP.

Advantages

- Lower latency
 - Better performance on unstable networks
 - Avoids transport-level Head-of-Line Blocking
-

16. Reverse Proxy

Client

↓

Reverse Proxy

↓

Servers

Responsibilities

- SSL Termination
- Compression
- Caching
- Security
- Load Balancing

Examples

- NGINX
- HAProxy

17. Important Terms

Term	Meaning
Latency	Time for one request-response
Throughput	Requests handled per second
Scalability	Ability to handle increasing traffic
Availability	System uptime
Reliability	Consistent correct operation
Fault Tolerance	Continues working after failures

18. Interview Flow (Most Important)

Browser

↓

DNS

↓

TCP Handshake

↓

TLS

↓

HTTP Request

↓

Load Balancer

↓

Application Server

↓

Cache

↓

Database

↓

Response

↓

Browser Rendering

19. Common Interview Questions

✓ What happens when you type google.com?

✓ Explain DNS.

✓ Why is HTTP stateless?

✓ HTTP vs HTTPS.

✓ Cookies vs Sessions vs JWT.

✓ What is TCP Three-Way Handshake?

✓ What is Cache?

✓ What is Load Balancer?

✓ Cache Hit vs Cache Miss.

✓ Why use HTTPS?

20. Common Mistakes

- ✗ Forget DNS lookup.
 - ✗ Forget TCP handshake.
 - ✗ Forget TLS handshake.
 - ✗ Say HTTP remembers users.
 - ✗ Ignore browser cache.
 - ✗ Ignore load balancer.
 - ✗ Forget browser rendering.
-

21. Memory Tricks

Request Flow

D → T → T → H → L → S → C → D → R

DNS → TCP → TLS → HTTP → Load Balancer → Server → Cache → Database → Response

Authentication

C → S → J

Cookies → Sessions → JWT

HTTPS

HTTP + TLS = HTTPS

Cache Rule

Cache First → Database Second

Scaling Rule

One Server? → Load Balancer → Multiple Servers

22. 30-Second Interview Answer

When a user enters a URL, the browser first checks its cache and performs a DNS lookup to obtain the server's IP address. It establishes a TCP connection and, for HTTPS, completes a TLS handshake. The browser sends an HTTP request, which is routed through a load balancer to an application server. The server checks the cache first; if the data is unavailable, it queries the database, prepares the response, and sends it back. Finally, the browser parses the HTML, CSS, and JavaScript, constructs the page, and renders it for the user.